Course summary

> Note that this summary is based solely on my own interpretation of the sources contained within the compendium. As a consequence of the age of the respective sources, it has been particularly difficult to comprehend the contents of the course. Not only because the sources read different, but also because some of the stem from an early era of CM where terms had not been agreed upon. As a result, it has been difficult to recognize when sources are corroborating on an already defined concept, and when they are talking about something else. View this document as a resource to check yourself against, not a complete source of knowledge. // Mikael

Software Configuration Management (SCM)

SCM benefits the development process in numerous ways. [24] It also increases business value, through various means: [25] - faster development means faster time to market [25] - better quality means less time spent debugging, more time spent enhancing [25]

It aims to maximize productivity, by minimizing the frequency of mistakes. [38]

Success factors

Milligan identifies several factors of success, such as: - Safety: the project should be securely accessed, e.g. only by those authorized, and it should be easy to recover from mistakes or hardware failures. [25] - Stability: the developer should have self-governance of what is in their ***workspace***; e.g. they know that when they go into work the next day, their files will look the same. Likewise, they can be assured that they only integrate upstream code when they desire and feel ready to do so. [25] - Control: the system should have a robust way to handle ***changes*** and ***parallel*** development, but simultaneously be flexible enough to encourage experimentation. [25] - Auditability: it should be possible to ***audit*** the system. Likewise, it should be possible to view the ***history*** of the repository and ***trace changes***. [25] - ***Reproducibility***: it should be possible to fully emulate an older ***version*** of the software. [25] More on this in the section on ***reproducibility***. - ***Traceability***: it should be possible to understand what went into the creation of a ***version***. [25] More on this in the section on ***traceability***. - Scalability: the system needs to be flexible enough to suit small and large teams, grow quickly, and support ***distributed teams***. [25]

Management

Roles within CM

Configuration manager The role of a configuration manager depends on the scope of their duties. If they are in charge of software, they may be regarded as software manager, and if they are in charge of documentation, they may be seen as a ***librarian***. [19, 24] The breadth of the managers responsibility depends on to what end they've been assigned. They can be assigned a whole division, or just a subproject. [19]

As is often the case, the role of software configuration manager is simply seen as a role of many assigned to the same person. It does not always make sense to have a dedicated configuration manager. Typically, this role is assigned to the software engineering manager. The usual prerequisites for the role are a background in software engineering or management, or both. The manager should be confident in the CM plan and it's contents, and vigilant about it's application, lest trust and application of the plan will weaken. [19]

It often makes sense to conduct configuration management on the global level, as confusion can occur on the project level if one team member is working on several projects with entirely different approaches to SCM. [19]

The configuration manager is typically very involved in, but not the chairman of, the ***CCB***. Their role typically comprises of reporting and providing insight. [19]

Planning

Configuration management plans (CM plans) An initial draft of the CM plan can be created and circulated among involved groups. Once feedback is obtained, it can be incorporated into the SCM plan to improve it. [18]

The CM plan is one other three keys to success in establishing good CM, amongst a CM system and adoption strategy. It is intended to be an accurate and complete description of all tasks and procedures, e.g. SCM functions, involved in configuring the product or system and the equipment and knowledge required to carry them out. [18]

The main motivation behind the creation of CM plans is that it increases ***awareness***, knowledge distribution, ***coordination***, and ***communication***. [18]

The creation of such a plan can be approached in several ways. Of great preference, particularly in large organizations, is the incremental approach, where SCM functions are gradually introduced until full SCM implementation is reached – however, this still requires some foresight, as the increments need to fit together. [18]

Several standards exist for CM plans, such as ones by IEEE, NASA, and DOD (now "DOW"). They all seek to identify the tasks within SCM, but do so at different levels of generalization. The ISO implementation in particular focuses more on the topics contained within this document, such as: ***configuration identification***, ***configuration control***, ***status accounting***, and ***configuration audits***. [18]

As is discussed later in this document, SCM plans are also subject to ***configuration auditing***. [17, 18] The quality of a CM plan can make or break the project. [18]

Leon, who bases his structure mainly on IEEE, suggests the following SCM plan framework: 1. Introduction 2. SCM management: organization, responsibilities, e.t.c. [18] 3. SCM activities: ***configuration identification***, ***configuration control***, ***change management***, ***status accounting***, ***configuration auditing***, e.t.c. [18, 20] 4. SCM schedules: sequence of SCM activities, identification of lifecycle and milestone, and or where different ***baselines*** will be established. [18] 5. SCM resources: identification of necessary tools, training, equipment, e.t.c. [18] 6. SCM plan maintenance: how to upkeep the SCM plan. [18]

Dart suggests a very similar model, with more granularity. [24]

Workshops The purpose of workshops are to involve developers in the construction of CM plans and processes. Secondarily, to increase their knowledge about CM practices. [2]

The pros include: - On the job training [2] - Development of company-wide CM plans and processes [2] - Spread of knowledge and experience between employees [2] - Increased morale, as employees feel personally responsible for the CM plan. [38]

It is conducted in three stages: 1. Prepare: select who should participate, when to conduct the workshop, and what background material needs to be made available [2] 2. Execute: present the objectives of the workshop and present the content (such as the three ***metaphors***) [2] 3. Document: draft the CM plan, review it, and publish it [2]

Activities

Bendix and Ekman / Leon identify four main activities of traditional SCM, being: ***Configuration Identification***, ***Configuration Control***, ***Configuration Status Accounting*** and ***Configuration Audit*** [18, 20]

Version management / Version control systems The means of storing, managing, and retrieving the history of ***configuration items***. Versions, revisions, derivations, and variants are identified and stored. [20] The ***configuration items*** themselves are the subject of ***configuration control / management***.

They are superior to a simple backup system, as they are more granular and therefore ensure ***reproducibility*** and ***traceability***. [38]

Versions Versions of the software. Not necessarily linear. Considered immutable and usually kept track of with version numbers. They are often ***baselined*** and released.

Revisions A version of the software intended to replace the old version in a linear fashion. The intent is that this version has fewer bugs or more functionality. They are numbered such that they can be recognized and retrieved if necessary. Intuitively numbered with ascending indices. [1]

Variants / Variations An alternative version of the software intended to be used as one of several options for configuring the software. Such as an enterprise vs. personal version of a program. [1, 24]

A set of them is called a variation set. [1]

The ***multiple maintenance trap*** arises when using variants. It is resolved by maintaining a good composition of ***artifacts***, such that variants are created through the customization of the few rather than the many – e.g., any code that can be made invariant between variants should be made invariant. In the cases where code cannot be shared between variants, there must be clear documentation and ***change management***. [21]

There are several approaches to variants, amongst them: - variant segregation: each variant has a separate copy of a component. [21] - single source variants: each variant is created by cherry picking parts of the repository during the build process. [21]

Variant segregation is usually chosen, but may not always be ideal, as it introduces redundancy and increases storage usage – in addition to the ***multiple maintenance trap***. The system can grow even more complex, if there is a need for subvariants, such as for each version of an operating system (for example one for Windows 7, 8, 10, 11, e.t.c.). [21]

Single source variants are popular within C, where it is achieved through conditional compilation. This approach avoids the redundancy and storage problems of the segregation approach, but it can become difficult to track the so-called meta programs. Unlike with segregation, it is hard to foresee what consequences a change will have on a particular meta program, as they are not explicit. As a subconsequence, this also means that ***traceability*** in the context of particular meta programs is reduced. [21]

There are many more types of variants as well, such as aggregate, ***derivation***, and temporary variants. [21]

There is a lot more to the field of variant management not covered here, in the interest of time.

Derivations The history of the software ***artifacts***. This is essential for maintaining ***traceability***, and particularly important for debugging. For the derivations to have any meaning, they must have identification and details. Such details include what tool with what options and input created an ***artifact***, why it was used that way, who did it, and when. Verbose derivations enable developers to quickly discover if the bug is the result of logical errors or setup issues. Documenting the derivations is also critical, such as providing changelogs or keeping copies of older versions [13] – something tools do for us these days. [0]

Build management The process of creating a runnable version of the system. Typically done automatically, and using the definitions within the ***system model***. [20]

Team co-ordination and communication Since communication cannot always be ensured, it is important that it is also carried out indirectly, such as through documentation or comments. [2]

Co-ordination strategies / Concurrency control *also known as concurrency control schemes* [9, 20]

Methods of facilitating or preventing simultaneous access. [22]

Turn-taking / Locking / Conservative scheme / Serial development

We simply lock the part of the repository that we are working on, such as a file or module, so no-one else can modify it. [7, 8, 9, 14, 20] Babich refers to the locking process as "charge-out", and the submission process as "charge-in". As with the ***checkout-check-in model***, it is important that the code being "charged in" is tested. [8] The obvious drawback is that it makes it impossible to work on tasks in parallel, [7, 20] and developers can forget to release their lock. [3] Usually, this strategy is motivated by a lack of confidence in the merge functionality. [7]

Split-combine Split the architecture into well-defined components. As a result, we can be relatively sure only one developer is working on any component at a time, negating the need for ***locking***. [7] However, it may still lead to situations where another developer accidentally uses a newer revision of a model when testing their code built for an older version. [8] The ***shared data*** and ***simultaneous update*** problems can still occur, particularly during the combine procedure. [20]

Copy-merge / Optimistic scheme We create a copy, edit whatever we need to, and hope no-one else has edited the same files. If they have, we trust the merge functionality to help us. [9] This usually works out fine, especially if the project is large in comparison to team size, so long as we keep the double maintenance problem in mind. [7]

Configuration control / management The process of regulating the implementation of approved ***changes***, their history,

and attribution, [24] as well as ***identifying*** and ***coordinating configuration items***, e.t.c.

Configuration identification The process of labelling or identifying all ***artifacts*** and selecting ***configuration items***. For some ***artifacts***, this may entail the application of serial numbers or other unique identifiers. [16, 20, 24] Usually done by a ***configuration manager***. Good configuration identification addresses ***traceability***. [16]

Configuration items *sometimes referred to as computer program / software configuration items (CPCI / CSCI), depending on what the configuration item is* [16]

Configuration items are defined as anything which can and should be independently identified and managed. [12, 35] They may be nested within each-other, however the important part is that it makes sense to group them as configuration items – e.g. the distinction should add value. The distinction should be made such that we would "mourn" it's loss (it would have consequences if anything happened to the configuration item) [12] – this is sometimes referred to as the "lowest replaceable unit (LRU)". [16] A line of code would not be categorized as one, but the source file might; or maybe it makes more sense to see the whole module as one?

Daniels proposes a recursive method of defining configuration items, where there are several levels to a hierarchy. What counts as a configuration item is at the eye of the beholder. [16]

Kelly further defines "stepping stone" items, which are interim items that are not worth storing, [12] such as testing results.

Configuration items are kept in the controlled area, sometimes referred to as a CM ***library (see metaphor)***. [12] Their status is tracked as a part of the ***status accounting*** process. [35]

They contain information such as status, attributes, and authorship / responsibility. [35]

Baselines A milestone in the development process, such as a release. It can be seen as a known good configuration, frozen in time such that it is ***reproducible***. It is used as a basis for further development, but is itself immutable. [26]

It contains details on configuration, such as which dependencies the project relies on.

Certain ***bound configurations*** can form a baseline, [2, 14] sometimes referred to as a baselevel [14] or approved configuration, [16] e.g. a basis for further development with formal ***change management***. [2]

Daniels identifies three primary forms of baselines: - functional baseline (FBL): what the system should do [16, 37] - allocated baseline (ABL): how system functions and requirements are allocated to components, e.g. how the system is decomposed [16, 37] - product baseline (PBL): the complete products requirements both on a technical level and a physical level [16, 37]

Software Bill of Materials (SBoM) A formal inventory of all dependencies and components in a project [28, 29, 30], usually those which are third party. The intention is to facilitate management of the software supply chain in order to increase security, track vulnerabilities, and comply with licenses. [29, 30]

It includes details such as supplier and component names, ***version*** used, unique identifiers, licenses, and more. [29, 30] It is not too dissimilar from a system model.

It may be directly included in the ***CI/CD*** pipelines, where the SBoM is automatically created through scanning the project at build time. [29, 30]

Configuration verification In configuration management, the verification process consists of audits and program reviews. [17]

Configuration audits There are various types of audits, roughly grouped into special and formal audits.

Configuration audits are a type of formal audit whose purpose is to verify that the product lines up with specifications and requirements from the technical documentation, and that the ***configuration item*** selection and naming convention is appropriate. [17, 24] Furthermore, it is checked whether the ***configuration item's*** relationship to the ***baseline*** is appropriate, and whether the ***status accounting*** system is valid. [17, 20]

There are two subtypes of configuration audits, namely Functional Configuration Audit (FCA) and Physical Configuration Audit (PCA). The first verifies functionality (ex. the ***functional baseline***), and the latter verifies form, fit, and function. FCA is often carried out during integration and qualification testing, whereas PCA is carried out during later evaluation. Typically carried out by a QA team. [17, 20]

In software applications, ***status accounting*** and auditing are grouped together, which Daniels sees as unfortunate. [17]

Conducting audits

An audit begins by defining what is to be audited, and the goals of the audit. In preparation, a so-called "data package" containing the relevant documentation needs to be collected. Secondarily, an audit team is gathered, including a chairperson and a recorder (note-taker). This team is often splintered during auditing. After conducting the audit, findings are collected, and a report is prepared. [17]

Reviews There are several types of reviews that may be carried out during specific parts of the lifecycle. These include: - System Requirements: Establish functional ***baseline***. ***CM plan*** complete, and procedure development begun. [17] - System Design: Establish the allocated ***baseline***, verify ***traceability*** back to top-level requirements. ***CM plan*** is reviewed and amended, and procedures (automated testing, e.t.c.) should be in place. [17] - Preliminary Design: Verify decomposition and ***configuration item*** selection. Review procedures, e.t.c. [17] - Critical Design: Establish developmental ***baseline***. Validate ***status accounting*** system. [17] - Product or Functional Qualification: Assure that product is ready for customer. [17]

Configuration status accounting *this seems to sometimes be referred to as status reporting.* [24]

A means of documenting the status and implementation state of ***configuration items*** [15, 16, 18, 20, 24, 33, 34] through their entire lifecycle. [33, 34] It also provides for ***communication*** and ***traceability***, [16, 18, 20] and provides updates on ***CCB*** activities. [16, 18] This may be carried out in the form of ***transaction*** logs, ***change*** logs, or progress reports. [20]

The aim is continually trace the development, such that all progress is attributed and dated [16] and able to retrieved and ***reproduced***. [3$]

Daniels outlines that an effective status accounting system satisfies: - is capable of handling many elements [16] - is flexible (can be adjusted to current needs) [16] - is easy to use [16] - can be controlled [16] - produces varied reports [16] - can account for security needs [16]

Change management Focuses on managing organizational changes. [36]

Change control The process of handling changes, particularly the approval process. [2, 17, 18, 20, 22, 24, 36] The control of changes to hardware, software, firmware, and documentation. [17] Somewhat intertwined with ***configuration control***, which tracks the implementation of approved changes.

Increases **traceability** and responsibility. [2, 3, 36]

It is not to be confused with change management, as it is a subset of it. [36]

Change control board

sometimes referred to as a change advisory board. [36]

The approval process is handled by a Change Control Board (CCB), [2, 17, 18, 24] whose members must be adequately knowledgeable about the software and product. [2] The board is made up of members who represent the major parts of the organization (such as engineering, production, e.t.c.). The CCB establishes ***baselines***, but does not carry out detailed technical ***reviews*** – these are carried out before the decision ends up by the board. [17]

The CCB includes a chairperson, secretary, members, and specialists. [17]

The change process

The CCB reviews the proposal both on the grounds of the necessity of the change, and it's impact and quality. The thoroughness of the implementation is assured through testing and peer review. As the size of the change grows, so does the complexity of the verification. Furthermore, depending on how the organization is structured, many parties may need to be involved in the process of approving a change. Due to the scope of the change process, certain urgency labelling may be required in some cases. [17]

A verdict is reached by the CCB. If it is declined, then it goes back the originator, e.g. the requester. Otherwise, it propagates up the chain of command, if required. [17]

While the process may look highly bureaucratic, the process increases ***traceability*** [2, 17], ***communication***, and information spread. [17]

SCM in other contexts

SCM vs. Product Data Management (PDM) Like SCM, PDM focuses on managing the design process. The difference is that SCM focuses on software, whereas PDM focuses on hardware. Many of the concepts overlap. [22]

PDM is, however, more concerned about managing data and documentation for hardware. There is a larger focus on re-use and product structure. Instead of managing source code, PDM often manages things like CAD files and 3D models. [22]

In some cases, SCM and PDM are used in unison, when a project requires both paradigms, though integration of them is not as simple as using both at the same time. [22]

They share, amongst other things: ***identification***, ***change management***, ***version control***, and ***variant management***. However, the particular ways they are approached may differ. [22]

SCM in Agile Development Agile methods embrace change and focus on how to respond rapidly to changes in the requirements and environment. The haste of the project often clashes with the bureaucratic nature of SCM – the responsibility often falls on the developers instead. It become more difficult to consistently adhere to procedures, reviews, and audits. [20]

In Agile, there is typically a large focus on test-driven development, ***automation***, refactoring, ***parallel work***, and ***continuous integration***. It also incorporates so-called planning games, which are informal and fast ways to continually define requirements. [20]

While the bureaucracy of SCM may sound counter-productive to agile, many of the procedures and principles are beneficial. [20]

Where agile largely fails in regards to SCM is in maintaining a good relationship between the team and the customer. [20]

Some tasks within SCM that are typically ignored or difficult to implement within agile are ***configuration identification***, ***configuration control***, ***status accounting***, ***audits***, ***roles***, ***tooling***, and ***CM plans***: [20] - ***Configuration identification***: may be resolved by defining rules for identification, rather than an initial structure. This removes the up-front cost of thoroughly defining all ***artifacts*** at the start. [20] - ***Configuration control***: while part of this task is handled through planning games or other agile activities, agile does not necessarily incorporate methods that preserve ***traceability***. In some cases, this is solved through having an explicit tracker role. [20] - ***Status accounting***: place relevant status information in the ***repository***. [20] - ***Configuration audits***: reduce ***audits*** to verification that ***audits*** have been carried out by QA. [20] - ***Roles***: since everyone is similarly involved in agile, it is important that everyone is similarly knowledgeable about SCM, as there is no room for dedicated SCM roles. [20] - ***CM plans***: the use of SCM processes at all is preferred to a comprehensive and monolithic SCM plan. Ideally, the processes are documented. [20]

SCM and DevOps Each team, or even every person, should be able to take care of any task. This means, like in ***agile***, there is no specialized SCM role. [23]

There is debate surrounding what DevOps exactly is, but Bendix, et. al. define it as a solution to the divide between Developers and Operations (IT support). If developers and operations are located in the same place and with good communication, conflicts and misunderstandings should decrease in frequency. [23, 32]

DevOps is useful in projects where there usually a divide between the consumers and the developers, e.g. where developers have little contact with the target user base. Thus, quickly gathering feedback from users is of great importance in DevOps. [23]

There is also a great focus on release quality, whilst releasing often. The main idea is that the equal competence of all developers enables faster processes and thereby faster releases. [23]

DevOps activities, such as ***continuous integration / continuous delivery (CI/CD)***, are supported by several facets of SCM, such as ***configuration control***, ***SCM plans***, ***status accounting***, ***version control***, ***communication***, ***configuration identification***, and ***change management*** – amongst many more. [23]

Parallel working, in general

Concurrency problems

Shared data problem Occurs when several developers are simultaneously accessing and modifying the same files. A modification made by one developer in one file may break functionality which another developer depended on. [1] One solution is the creation of ***private workspaces***. [20] It is impossible to fully resolve, even if there is only one developer, as even they can end up writing dependency-breaking code. [38]

Double maintenance problem When several versions of the same software are being maintained independently. If a modification, such as a bug fix, occurs in one version, then that modification has to be manually transplanted to all other versions. At worst, developers may independently fix the same bug twice, maybe in different ways. Over time, this can lead to divergence between the versions. [1, 20] This problem is independent of the amount of developers, as any situation where several copies co-exist will cause the problem. [38] This is resolved by careful merging rules and ***co-ordination***.

Simultaneous update problem When several developers simultaneously update the same files, they can overwrite each-others changes by accident, which can lead to resolved bugs reappearing or added functionality disappearing. [1, 20] This may be solved by assigning identification, such as ***versions*** to the files, to enable recognition of desynchronization [20] – e.g. through ***long or strict long transactions***. More on this in the section on ***co-ordination strategies***.

Multiple maintenance trap Very related to the ***double maintenance problem***. Occurs in an environment with ***variants***, where each ***variant*** may be customized to fit a particular need. Since ***variants*** are not necessarily linked to each-other, but descendants of the same ***version***, fixes in one ***variant*** need to be manually applied to each relevant ***variant***. [21]

Distributed development

Occurs when developers are physically, particularly geographically, separated. [4, 5] An increasingly common practice, as it allows companies to reach talent in different locations (Global Software Development). [5]

Challenges Challenges include decreased communication, reliance on the internet, and implementing CM practices. It can also lower team morale and knowledge spread. [4]

- Distributed groups usually do not get information on what other groups are doing, [4, 5] e.g. the group ***awareness*** is lowered [5]

  Strategies: Have well defined tasks and clear areas of responsibility. [5]

- Merge requests usually take longer, due to timezones and lowered group ***awareness***. The ***CCB*** may not recognize some of the code submitted, which creates research overhead. [5]

  Strategies: Improving communication and documentation. Having well defined tasks and clear areas of responsibility.

- Different SCM environments lead to merge requests being handled differently [5]

  Strategies: Make sure all members adhere to the same ***CM plan***. [5]

- Lack of a planned ***baseline***. [5]

  Strategies: Establish ***baseline*** before development starts, and make sure all developers have adequate knowledge about it. [5]

- Lack of SCM principles [5]

  Strategies: Implement them... [5]

- Tasks are not always clearly distributed [6]
- Lowered communication [6]

Cases of distributed development From an SCM perspective, there is not difference between parallel work and distributed development. [20]

Locally All developers are on-site in the same location. [4] - Common file system [4] - Complete development and test environment [4] - Synchronization can be achieved through meetings, but also opportunistically [4] - Less security issues, as less services are exposed [4]

Distance working Some developers are working off-site, such as from home. This is either achieved by allowing developers to bring a copy home or by letting them remote login to the worksite. [4] - Poorer communication, less opportunities for spontaneous synchronization [4] - More security issues, as more services need to be exposed [4]

Outsourcing Some components of the system are developed by third parties, who work in a different location. [4] - Requires clear instructions from the purchaser [4] - Supplier must be able to update the development and test environments [4] - The purchaser and supplier may not use the same tools, particularly CM and build tools, which may complicate integration [4]

Co-located groups Work is performed in groups, but the groups are in different locations. Such as different divisions of a company. [4] - Different file systems, hopefully same CM tools [4] - The groups deliver subproducts to each-other rather than developing together [4] - Synchronization is achieved solely through planned meetings [4] - ***Change management*** becomes particularly important [4]

Distributed groups Work is performed in groups. Group members may be spread geographically. Similar to the case of distance working. Can usually be avoided by subgrouping. [4] - Communication becomes very important [4] - Simultaneous access must be possible [4] - Solutions using locking work poorly [4]

White's three scenarios Multiple teams: Producer/consumer

Teams are geographically distributed and share components in a producer/consumer relationship. The producer develops the component, and the consumer uses it. However, the consumer does NOT modify it. [6]

Can sometimes be an example of ***outsourcing***, but often it could be ***co-located groups***.

White argues that this decreases the amount of integration problems. It is necessary that the component breakdown makes sense, so foresight is necessary. [6]

This presents some challenges: 1. you'll need to identify the versions of each component and establish component ***baselines*** [6] 2. components need a reliable way of being delivered [6] 3. components need to be structured in advance, such as through stubs [6]

Multiple teams: Shared source code

Teams are geographically distributed, but modify the same shared project. [6]

White advises to avoid this as much as possible, unless, in one of these (non-exhaustive) situations: - Monolithic architecture [6] - System is in maintenance mode [6] - Several teams NEED to access it simultaneously [6] - Organization has a feature-based approach, e.g. developers modify any code relevant to the feature they've been assigned [6]

Single team: Distributed members

An example of the distance working or distributed groups cases.

Members are geographically distributed, but not organized into teams. They modify the same shared project. [6]

Architecture and management occurs at the site where most developers are located. Like the previous examples, the system is divided into components. In this case, components are assigned to individuals rather than teams. Shared code is inevitable, though. [6]

Architectures One server / site

- *Locally to a server*: All developers are in the same location, working on the same server, [4] such as a CVS server.
- *Remote login*: Some developers are in the same location, some are distributed. The developers remote to the same server. [4, 6] The protocol used to achieve this may vary. If HTTP/HTTPS is used, it is regarded as web access. [6]

Several servers / sites

- *Master-Slave connections*: One site is considered "Master", and is synchronized to by the "Slaves". It is important that both sites use the same CM tooling. [4]
- *Areas of responsibility*: Each site is responsible for a part of the project. Essentially, piece-wise "Master-Slave". [4]
- *Equal servers*: All sites are synchronized with each-other and act like one logical site. Developers remote into the logical site, and get connected to whichever site makes most sense given their connection [4]

Other

- *Local access*: Developers make a copy to bring home. [4, 6]
- *Disconnected access*: Developers login to the server to make a copy, then disconnect. Similar to local access, just the transfer process is different. [6]

Metaphors

Construction site metaphor

In a construction site, people of varying trades need to work simultaneously on the same project. This can cause situations where some contractors have to wait for others to finish. Without foresight, it can be difficult to work in unison.

In the context of software, this is equivalent to a project with several developers. To work on the software simultaneously, the developers need their own copies of the ***artifact***. This immediately presents issues in the form of the ***Double Maintenance***, ***Shared Data***, and ***Simultaneous Update*** problems. Again, parallelization is difficult without foresight. [2]

The easy solution is to serialize, the obvious downside being decreased productivity. The other option is to maintain good ***communication*** in addition to a ***change management system***. [2, 3] One option is ***locking***, which presents it's own issues. [3]

It is also important that ***build processes*** and tooling are consistent. [3]

Study metaphor

In a study, you bring in your study materials (books, e.t.c.), which you work with and produce new works, such as notes. You do so undisturbed, on your own. [2]

In the context of software, this is equivalent to the ***checkout/check-in model***.

Library metaphor

We keep knowledge in libraries, organized in such a way that it is easy to retrieve and store information. The library also needs to manage access, such as through loans. [2]

In software, this is comparable to tracking ***artifacts*** involved in the project. We need a way to store, recreate, and register the history of an ***artifact***. [2] SCM facilitates this through whats often referred to as a ***CM library***.

Solutions and strategies

Automation

We can automate trivial and repetitive tasks. An early example of this is `make`. In the present day, we have even more advanced tools with system models and build processes. [7]

Continuous Delivery / Continuous Integration (CI/CD)

in some sources, continuous delivery is referred to as "regular builds" [20]

A process which facilitates the ability to release at any point, negating the need for timed releases. [10, 11] At each commit, the repository is rebuilt, tested, and deployed. [11] Through the continuous integration of changes, we partially avoid the ***double maintenance problem***. [20]

When releasing on regular schedules, shippable features may be lost as the cut-off approached. This means that a feature completed in, say, week 7 may not be released until week 16. Continuous delivery avoids this problem, as the feature would simply be released when it is complete. The constant feedback cycle also means that problems arise quicker, rather than several weeks after the feature was supposedly complete. [10] As the testing process is made as automatic as possible, it also drastically decreases the time required to test. Considering the frequency of tests, releases end up more reliable as well. [11]

On the business end, it also allows us to keep up with competition. Instead of releasing a competitive version of the software next release cycle, we can release it as soon as possible. [10, 11] It also allows us to weed out unwanted features. [11]

Integrating continuous integration into the workflow presents challenges, however. A lot of things need to be automated, and it

may take time. A good approach is to incrementally automate the project. Integration itself needs to be made more efficient as well; this could be achieved by using fewer ***branches*** (if any), and small features and commits. Automated, reliable, fast, and credible testing is likewise incredibly important. [10, 20] Some corners may also have to be cut, such as the traditional ***change request board***. [11] ***Functional audits*** may be reduced to acceptance tests. [20]

Neely and Stolt suggest using feature toggles. That way, features can be rolled out selectively, and rolled back at request. When releasing, the necessary features can be toggled, [10] similar to the ***change-set model***. This enables another similar solution, so-called "canary deployments", where new features are released to a small portion of the user base first.

Models and patterns

Models

Checkout/check-in model A separation of where files are stored, and where they are modified. The developer "checks out" a copy of the repository, and "checks in" when they are done. [7, 9, 14] This solves the ***Shared Data problem***, but can cause the ***Double Maintenance problem***. The ***Simultaneous Update problem*** may also occur if someone checks in a modification to the same file as the developer intends to update between checking out and checking in. To avoid the ***Simultaneous Update problem***, we need to provide ***versioning*** support. To facilitate the check-in process, we also need merge functionality and it may be advantageous to provide branching support. [7]

The composition model Similar to the ***checkout/check-in*** model, we use ***repositories*** and ***workspaces***, in addition to ***concurrency control*** through ***locking***. Here, however, the configuration is based on system models and version selection rules. [9] The selection rules may come in the form of ***bound configurations***.

This enables ***reproducibility***, but introduces complexity and sometimes inflexibility.

System model Lists the component that make up the system. Relies on ***version selection rules***. [9]

Version selection rules The version selection rules indicate which versions of the dependencies and modules are being used in a system configuration. [9]

Bound configurations

Versions are static, e.g. specific versions of components are used. They can be used as baselines. [2, 9] Generally more stable and ***reproducible***. [9]

Partially bound configurations

Versions are relative, e.g. "use latest version of", "version < 2.9", e.t.c. [2, 9] This can, of course, produce instability, as the rules may accidentally include versions with breaking changes. [9]

The change-set model Individual modifications are grouped into identifiable ***change sets***, such that they can be arbitrarily applied to a workspace. [9, 24] This links naturally with the concept of ***change requests***, as a ***change set*** could become a ***change request***. The history is stored as a chain of ***change sets***, rather than ***versions***. It lends itself to increased ***traceability***, as individual changes can be traced. [9]

Change sets are similar to ***long transactions***, but differ in that the developer is able to resolve some of the conflicts, and changes are seen at the line level rather than the file level. ***Configurations*** can in this case be seen as a ***baseline*** in conjunction with a list of ***change sets***, applied depending on desired configuration. [9]

Two-tier model Development is separated into two tiers; formal configuration control, and development (version control only). Developers interact with what amounts to the ***checkout/check-in*** model. Once their contributions are ironed out, they escalate to the formal configuration control. Developers are thus less burdened by overhead, and we get to maintain SCM practices. [12]

A ***configuration item*** escalates once it has been tested or when it is acutely needed for review or use. If the latter is the case, it must be made abundantly clear. [12]

The limitations to this model used to be storage, in which case we could compress the files, [12] but this is rarely ever a problem anymore. [0]

An extension of this, the three-tier model, can be seen as the addition of a ***CM library*** tier. [38]

Distributed vs. Centralized model In a centralized model (CVS, Subversion), all code is stored on the same server, and developers work directly of it – their commits are saved immediately onto the server. A distributed model (Git, Mercurial), by contrast, facilitates the creation of ***workspaces***, which allow developers to work independently and integrate when they please. The prior means increased control and ***traceability***, but the latter is more flexible. [31]

A centralized model creates a single point of failure, whereas distributed models do not have ***locking*** – which may be desired in some cases. [31]

Patterns

- Organizational: How the organization is structured; size of team, management style, e.t.c. [14]
- Architectural: How the software is structured. [14]
- Process Defining: Structures, such as directory hierarchy. Things that are defined at the start of development. [14]
- Maintaining: Day-to-day processes at the organization. The line between this pattern and process defining is slightly blurry. [14]

The Five Dimensions

The following dimensions can be used in isolation, or combined.

Bendix particularly wants us to understand how they are related to the course, which is why I have not merged this into the correct parts of the summary.

Version Each step of development warrants a new version. It is important that versions do not overwrite each-other, it should be possible to ***reproduce*** earlier versions. [15] This is directly comparable to our other definitions of ***versions***.

Views The development is divided into constituent and sequential parts of a process. For example, source code is made into compiled code, sketches are made into blueprints, e.t.c. Views encapsulate the steps contained within the development of one aspect of the project. [15]

Hierarchy Development is subdivided into subtasks, until it is made digestible. Another benefit is that if said deliverables are common within the project, such as a helper function, it can promote **reuse**. [15] It could be seen as a further compartmentalization of the view dimension.

Status Tracking the status of different tasks, such as whether something is ready for testing [15] – e.g. ***status accounting***.

Variants The reconfiguration of the product to fit many purposes, such as making the software run on different operating systems. [15, 21, 24] They are essentially parallel ***versions***. [21] This is treated the same way we have defined ***variants*** previously.

Glossary

What follows is a general set of terms, that didn't quite fit under any other umbrella.

Awareness

The collective understanding of the project within the team, particularly in regards to who is doing what. [0, 5, 7] One facet of this is provding versioning and a way to see the difference between two versions of the software, such as through a command like `diff`. [7]

Artifact

Anything created during development, such as a piece of code or documentation. [2] They are not necessarily the subject of any formal ***configuration control***, but can be if desired. A formalized artifact is often regarded as a ***configuration item***. [38]

Library

Artifacts or ***configuration items*** are stored in a library. This includes both physical and digital items, [12] with the latter being more common today. Like real libraries, items stored within them have permissions associated with them. [12]

Traceability

The ability to trace when changes were made, and by who. [14, 20, 25] Additionally, also exactly what configuration was used to make a certain build or release. [20, 25] Enables reusability. [14]

The intent is to increase ***co-ordination*** and improve impact analysis processes. [20]

Reproducibility

We need to be able to recover the complete configuration at any point in time, such that we can mimic the exact environment at that time. [13, 14, 25] This is critical for ***traceability*** and the ability to debug previously released versions. For reproducibility to work, ***derivations*** must be immutable. [13, 14]

Transactions

A transaction is a unit of work, such as a modification of a file. The term is inherited from database theory. [7, 9]

The long transaction model A chain of several modifications, to several files, in a ***workspace*** – a "logic change". When we try to commit, a ***concurrency control*** scheme is executed. [9] Usually, this means that the ***version control system*** will check if we are up-to-date on the files we have updated. If not, it forces us to pull the latest changes and merge them into our ***workspace***. [7]

The strict long transaction model The usual long transaction model can cause issues, since the sum of parts in this case may be lesser than the whole – e.g. the new merged version may not even work even if the merge was successful. As such, strict long transactions enforce the constraint that all changes must be integrated, even changes to files we have not worked on. [7] Enforcing them can cause it's own headaches, such as slower development.

Repositories

sometimes referred to as Version Object Bases (VOBs) [38]

A shared project database, containing all ***artifacts*** / components of the the software. [8, 20] It's main responsibility is to co-ordinate ***parallel work***, through the facilitation of ***workspace*** creation and ***version control***. Most repositories are able to handle ***parallel work*** without the need for ***locking***. [20]

It is where known good code goes. A developer should not commit from their workspace any faulty code. [8] The repository is responsible for ***versioning*** [9] and needs to facilitate ***configuration management*** in order to facilitate consistency between ***workspaces***. [2]

Repositories are considered immutable, and developers are required to make a copy to work on it. [2]

Private workspaces

Each developer has their own private workspace, a long-lived and isolated environment, containing a copy of the ***repository*** and current changes. When the developer is done, and has tested their code, they can contribute it to the ***repository***, [8, 9, 20] as in the ***checkout/check-in model***.

Commits

Snapshot of changes made in the ***workspace***. [38]

Branching

We create a copy of the ***repository*** at a certain point in time and give it a new identifier. The development on the "main" branch continues as normal, while the new branch acts like a secondary ***repository*** from which another set of developers can work on. This may be used in conjunction with ***locking***, where the branching concept provides the ability for a developer to branch the ***repository*** to get around someones ***lock*** safely. [14]

When changes within a branch need to be integrated into another, we merge. This is sometimes referred to as syncing or propagating. [14]

Branching can exist on several levels. We may just branch the code, or we may branch the entire configuration. In some cases, we may branch the team itself. [14]

Branches may visually be represented as tree diagrams. [14]

Concerns within branching include safety, liveness, reusability, ***teamwork***, and SCM tool support. [14]

Branch creation patterns *only including the ones we needed to read about!* 1. Branch per Task (C2) Create a branch for each task. [14] 1. Branch per Major Task: create a branch when implementing very large features or bug-fixes [14] 2. Branch per ***Merge / Change request*** [14] 3. Personal activity branch: pet branch used by one developer [14] 2. Codeline per Codeline (C3) Use a seperate branch for each major and minor release [14] Codeline usually means branch. [27] 3. Subproject Line (C4) Create a branch for a set of tasks. [14] 1. Personal: again, a pet branch [14] 2. Experimental: a pet branch for several developers [14] 3. Multi-project: separating components into branches. Branches are used together when creating release build. [14]

Branch policy patterns *only including the ones we needed to read about!* 1. Merge Early and Often (P5) Merge contributions as soon as they are satisfiable and tested [14] 2. Early Branching (P7) Create the branch as soon as it is needed, don't dwell on it. [14] 3. Deferred Branching (P8) Do not create a branch unless it is strictly needed, such as when it starts conflicting with parallel work. [14]

Branch structuring patterns *only including the ones we needed to read about!* 1. Parallel Maintenance / Development Lines (S2) Split maintenance and development into separate branches. Maintenance merges into the development branch. [14] (is that not backwards?) 2. Staged Integration Lines (S5) Using several branches to represent a hierarchy, such as alpha -> beta -> development. Features work themselves upward in the hierarchy until ready for release. [14] 3. Change Propagation Queues (S6) Define relationships between branches such that changes are propagated in the order they were made. [14]

Sources

1-25 are part of the compendium. The rest have been found as part of my own research.

Sources are not thoroughly cited according to any format. It is sloppy, but I hope you can find the original sources regardless.

0. Own intuition or experiences
1. Babich chapter 1-2; https://archive.org/details/softwareconfigur0000babi/page/2/mode/2up
2. Bendix, Vinter
3. Mikkelsen, Pherigo
4. Asklund
5. Fauzi, et. al.
6. White
7. Bendix (the study metaphor)
8. Babich chapter 5; https://archive.org/details/softwareconfigur0000babi/page/2/mode/2up
9. Feiler
10. Neely, Stolt
11. Chen
12. Kelly
13. Babich chapter 3; https://archive.org/details/softwareconfigur0000babi/page/2/mode/2up
14. Appleton, et. al.
15. Van den Hamer, Lepoeter
16. M.A. Daniels, chapters 3-4
17. M.A. Daniels, chapter 2 and 5
18. Leon
19. Compton, Conner
20. Bendix, Ekman
21. Mahler
22. Crnkovic, et. al.
23. Bendix, Pendleton, et. al.
24. Dart
25. Milligan
26. https://www.perforce.com/blog/alm/best-practices-baselines
27. https://perforce-user.perforce.narkive.com/W3G1i0Rn/what-is-a-codeline
28. https://www.cisa.gov/sbom
29. https://www.ibm.com/think/topics/sbom
30. https://www.blackduck.com/blog/software-bill-of-materials-bom.html
31. Version Control Systems in Corporations: Centralized and Distributed, Högblom & Green: https://gupea.ub.gu.se/bitstreams/7f0e3f25-89df-4edb-a59b-3479ad8ca748/download
32. https://www.atlassian.com/devops
33. https://docs.microfocus.com/SM/9.50/Hybrid/Content/BestPrac
34. https://www.globalspec.com/reference/71660/203279/5-4-configuration-status-accounting
35. https://www.ibm.com/docs/en/control-desk/7.6.1?topic=overview-configuration-items
36. https://www.atlassian.com/itsm/change-management/change-control-process
37. https://acqnotes.com/acqnote/careerfields/configuration-baselines
38. Other students' resources, which I cannot share without permission